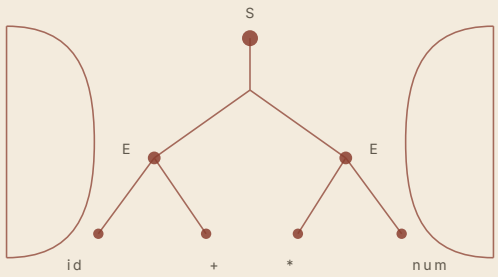
BMÜ325 · PROGRAMLAMA DİLLERİ

Önemli Dillerin Gelişimi

Dillerin soyağacı, tasarım amaçları ve değerlendirmeleri

Plankalkül'den C#'a · Dillerin tasarım gerekçeleri

BÖLÜM 2 · PD2



Ders slaytlarından derlenmiştir: PD2 (103 slayt).
Metin içermeyen eğitsel slaytlar görsel olarak incelenmiştir.
Ek açıklamalar "Açıklayıcı not" etiketiyle işaretlenmiştir.

ÖN KISIM · OKUMA NOTU

Bu kitapçık hakkında

Bu kılavuz, PD2 ders destesinin sınav odaklı, kâğıt hissi veren bir çalışma özetidir. Amaç slaytları tekrar etmek değil; her dilin hangi soruna yanıt olarak doğduğunu, tasarım amaçlarını, getirdiği yenilikleri ve sınırlarını tek bir metinde toplamaktır.

KAYNAK	PD2 “Önemli Programlama Dillerinin Gelişimi” (103 slayt). Kapsam 2.1–2.20 arasındaki yirmi alt bölümdür.
SAYFA DÜZENİ	Kaynak sırası korunmuştur; referanslar [PD2, s. 7] biçimindedir. Slayt numarası ile PDF sayfası aynıdır.
YÖNTEM	Metin katmanı PDF’den çıkarıldı; metin içermeyen eğitsel slaytlar sayfa görüntüsü olarak okundu.
DOĞRULUK	Kaynakta bulunmayan ek bilgi yalnızca **Açıklayıcı not** etiketiyle verilir. Kaynak içi tutarsızlıklar sessizce düzeltilmez.
SÜRÜM	PD2 tek başına; aynı şablon PD3–PD8 için de kullanılacaktır.

KILAVUZUN EKSENİ

Tarih, rastgele bir dil listesi değildir. Her dil belirli bir donanım, uygulama alanı ve tasarım anlayışının ürünüdür: Fortran sayısal hesaplamanın, COBOL ticari kayıtların, LISP sembolik yapay zekânın, ALGOL 60 taşınabilir algoritma dilinin, SIMULA 67 benzetimin, Smalltalk nesne soyutlamasının, Java taşınabilir ağ programlarının yanıtıdır. Bölüm boyunca üç soru izlenir: **hangi sorun**, **hangi tasarım kararı**, **hangi bedel**.

KAYNAK VE KAPSAM

PD2, Roberto Sebesta’nın *Concepts of Programming Languages* çizgisindeki tarih anlatımını izler; kaynakçada Sebesta (2013), Watt (2004), Scott (2009) ve Türkçe ders notları sayılır [PD2, s. 103]. Kılavuz yalnızca slaytlardaki bilgiyi düzenler; slaytta bulunmayan tarih, sürüm veya özellik eklenmez. Kaynakta tek cümleyle geçen bir dil, burada bağlamı ve sınavda ayırt edici yönü kadar açılır.

KISALTMALAR

PD programlama dili ders destesi · **BNF** Backus–Naur formu · **CFG** bağlam-bağımsız gramer (context-free grammar) · **OOP** nesneye yönelik programlama (object-oriented programming) · **ADT** soyut veri tipi (abstract data type) · **JVM** Java Sanal Makinesi · **CLR** Common Language Runtime · **CTS** Common Type System · **DoD** ABD Savunma Bakanlığı (Department of Defense) · **AI** yapay zekâ (artificial intelligence) · **GUI** grafik kullanıcı arayüzü.

İÇİNDEKİLER

Kılavuzun haritası

Sayfa numaraları bu PDF'e aittir; kaynak slayt referansları metin içinde verilmiştir.

A · ÇERÇEVE

Kılavuzun ekseni 2

B · AYRINTILI İNCELEME

Bölüm 2 — Dillerin gelişimi 4

2.1 Zuse'nin Plankalkül'ü 4

2.2 Minimum donanım programlama: sözdekodlar 5

2.3 IBM 704 ve Fortran 5

2.4 Fonksiyonel programlama: LISP 7

2.5 Sofistikeliğe doğru ilk adım: ALGOL 60 7

2.6 Ticari kayıtları bilgisayara uyarlamak: COBOL 8

2.7 Zaman paylaşımının başlangıcı: BASIC 9

2.8 Herkes için her şey: PL/I 9

2.9 İlk iki dinamik dil: APL ve SNOBOL 10

2.10 Veri soyutlamanın başlangıçları: SIMULA 67 10

2.11 Ortogonal tasarım: ALGOL 68 11

2.12 ALGOL'lerin ilk torunları 11

2.13 Mantık temelli programlama: Prolog 12

2.14 Tarihin en büyük tasarım çabası: Ada 12

2.15 Nesneye dayalı programlama: Smalltalk 13

2.16 Zorunlu ve nesneye dayalı özellikleri birleştirmek: C++ 13

2.17 Zorunlu tabanlı nesneye dayalı dil: Java 13

2.18 Ağ için betik dilleri 14

2.19 Yeni milenyum için C temelli dil: C# 15

2.20 İşaretleme/programlama hibrit diller 15

C–G · ORTAK BÖLÜMLER

C. Kavramlar ve tanımlar 17

D. Yöntemler ve süreçler 18

E. Formüller, kurallar ve olgular 19

F. Bağlantılar ve karşılaştırmalar 21

G. Kapsam denetimi 22

B · AYRINTILI İNCELEME
KAYNAK: PD2 · 103 SLAYT

BÖLÜM 2

Önemli Programlama Dillerinin Gelişimi

İlk programlama dili ENIAC ile mi doğdu? Plankalkül neden hiç uygulanmadı? Fortran neden makine kodundan daha yavaş olduğu hâlde kazandı? ALGOL 60 teknik olarak çığır açarken ticari olarak neden başarısız oldu? Bu bölüm, dilleri kronolojik sırayla ve tasarım amaçları üzerinden izler.

Kaynak aralığı: PD2, s. 1–103 · Alt bölümler: 2.1–2.20

BÖLÜMÜN HARİTASI: ENIAC'TAN C#'A

Kaynak, tarih anlatımına **ENIAC** ile başlar. 1943'te ENIAC ile birlikte gelen ENIAC Programlama Dili, yalnızca matematiksel işlemler yapabilen ve ikili sayı sistemiyle (0–1) çalışan devrim niteliğinde bir dildi; delikli kartlardan okunan ikili veriyle işlem yapıyordu ve programlama mantığı bugün de geçerli olan bir mimariye dayanıyordu [PD2, s. 5–6]. Bu başlangıç, sonraki bölümlerin ortak sorusunu kurar: **makineye yakınlık ile insan üretkenliği arasında denge.**

DÖNEM	TEMSİLCİ DİLLER	BASKIN SORU
1940'lar	Plankalkül, Short Code, Speedcoding	Makine kodunun okunaksızlığını azaltmak
1950'ler	Fortran, LISP, ALGOL 58/60, COBOL	Bilimsel, sembolik ve ticari alanlara ayrı diller
1960'lar	BASIC, PL/I, APL, SNOBOL, SIMULA 67	Zaman paylaşımı, çok amaçlılık, dinamiklik, benzetim
1968–1980	ALGOL 68, Pascal, C, Prolog, Ada, Smalltalk	Ortogonallik, yapısal öğretim, sistem programlama, mantık, gömülü sistem, OOP
1980 sonrası	C++, Eiffel, Delphi, Java, betik dilleri, C#	OOP'yi zorunlu dillerle birleştirmek, taşınabilirlik, web, bileşen

> **AÇIKLAYICI NOT**
> Tablodaki dönem ayırıcıları kaynak slaytların sırasına göre düzenlenmiştir; kesin tarih sınırları değildir. Bir dilin ilk tasarımı ile yaygınlaşması arasında yıllar olabilir (Ada'nın ilk kullanılabilir derleyicisi tasarımın bitiminden yaklaşık beş yıl sonra gelmiştir [PD2, s. 76]).

2.1 Zuse'nin Plankalkül'ü

Plankalkül, Konrad Zuse (1910–1995) liderliğinde Almanya'da geliştirilen ve **ilk yüksek düzeyli programlama dili** kabul edilen dildir. Donanımın ayrıntılarına bağlı olmayan bu dil, ENIAC dilinin hantallığına karşı bir atılımdı; Almanya'nın resmî programlama dili olarak nitelendirilebilecek biçimde tasarlandı ve istenen işlemleri ENIAC'a göre daha hızlı yapabildi [PD2, s. 7–8].
Dilin ayırt edici yanı, bugünkü dillerde standart olan veri yapılarını çok erken öngörmesidir: **kayan nokta (floating point)**, **diziler (arrays)** ve **kayıtlar (records)**. Ayrıca **sabitler (invariants)** kavramı yer alır. Buna karşın dil asla tam olarak geliştirilmedi; ENIAC ekibinin daha avantajlı konumu ve hızlı yanıtı, Plankalkül'ün duyulmasını engelledi [PD2, s. 7–8].

GÖSTERİM BİÇİMİ

Plankalkül'ün sözdizimi, bugünkü satır içi gösterimden farklıdır. `A[4] + 1` deyimini `A[5]`'e atayan ifade şöyle yazılır [PD2, s. 9]:

```
| A + 1 => A
V |  4       5     (altsimgeler – subscripts)
S |  1.n     1.n   (veri tipleri – data types)
```

`V` satırı altsimgeleri, `S` satırı veri tiplerini gösterir. Bu düzen, programı kâğıt üzerinde iki boyutlu bir tablo gibi okutur; çağdaşı dillerde görülmeyen bir soyutlama düzeyidir.

NEDEN GEÇ KALDI?

Plankalkül kapsamlı biçimde ancak **1972**'de yayımlandı; ilk derleyicisi **1998**'de gerçekleştirildi, bağımsız bir uygulama **2000**'de Berlin Hür Üniversitesi'nde izledi. "Kalkül" resmî sistem anlamına gelir; Hilbert tarzı ispat sistemi başlangıçta "Hilbert-Kalkül" olarak adlandırıldığı için Plankalkül "planlamak için resmî sistem" demektir. Dil bugün geniş kitlelerce öğrenilebilir bir dil değildir; ancak Almanya resmî dilini yaşatmaya çalışmıştır [PD2, s. 10].

> **KRITIK AYRIM**
>
> **İlk tasarlanan** dil ile **ilk uygulanan** dil aynı değildir. Plankalkül kavramsal olarak Fortran'dan öncedir; fakat derleyicisi 1998'de çıktığı için pratikte Fortran ilk yaygın yüksek düzeyli dil olmuştur. Sınavda "ilk yüksek düzeyli dil hangisidir?" sorusunun yanıtı Plankalkül; "ilk ticari/yaygın derlenen dil?" sorusunun yanıtı Fortran'dır.

2.2 Minimum donanım programlama: sözdekodlar

Makine koduyla program yazmak neden yanlıştı? Kaynak dört gerekçe sıralar: **az okunabilirlik**, **az değiştirilebilirlik**, **deyim kodlamanın usandırıcılığı** ve makinede **indeksleme veya kayan nokta** gibi donanım eksiklikleri [PD2, s. 11]. Sözdekodlar (pseudocodes), bu sorunları tam bir derleyici kurmadan hafifletmeye çalışan ara çözümlerdir.

SHORT CODE

Short Code, 1949'da Mauchly tarafından BINAC bilgisayarları için geliştirildi. Deyimler soldan sağa doğru kodlanıyordu. İşlem kodlarından bazıları şunlardır [PD2, s. 12]:

```
01  abs value          06  (n+2)nd power
02  )                  07  + 2n (n+2)nd root
03  =                  08  pause
04  /                  09  4n if <= n
05  (                  58  print and tab
```

Bu kodlar, makine komutlarının insan tarafından hatırlanabilir kısa adlarıdır; amaç programcının sayısal makine kodunu doğrudan yazmasını azaltmaktır.

SPEEDCODİNG

Speedcoding, 1954'te Backus tarafından IBM 701 için geliştirildi. Aritmetik ve matematiksel fonksiyonlar için sözde işlemler, koşullu ve koşulsuz dallanma, dizi erişiminde kaydedicileri otomatik artırma sağlıyordu. Ancak **yavaştı** ve kullanıcı programı için yalnızca **700 kelime** ayrılmıştı [PD2, s. 13]. Bu sınır, sözdekodların neden genel amaçlı bir dile dönüşemediğini gösterir: yorumlama/çeviri maliyeti, kazanılan okunabilirliği gölgeleyebiliyordu.

İLGİLİ SİSTEMLER

UNIVAC Derleme Sistemi, Grace Hopper yönetimindeki bir ekip tarafından geliştirildi ve sözdekodu makine koduna genişletti. David J. Wheeler (Cambridge Üniversitesi) ise **salt adresleme (absolute addressing)** problemini çözmek için **yeniden yerleştirilebilir adres blokları** kullanan bir yöntem geliştirdi [PD2, s. 14]. Bu iki katkı, sonraki derleyicilerin iki temel sorununu önceden kurar: **kod genişletme** ve **adres yeniden yerleştirme**.

> **AÇIKLAYICI NOT**
>
> "Sözdekod" bugünkü anlamıyla bir dil değil, bir **çeviri stratejisidir**: insan için okunabilir kısa komutları makine komutlarına açan bir katman. Short Code ve Speedcoding'in sınırlı belleği, bu stratejinin yalnızca küçük programlarda yaşayabildiğini gösterir.

2.3 IBM 704 ve Fortran

Fortran, **Fortran 0** ile 1954'te IBM'de John Backus tarafından tasarlandı; ancak bu sürüm uygulanmadı. **Fortran I**, 1957'de ticari kullanıma sunuldu ve **IBM 704** için tasarlandı. 704'ün **indeks yazmaçları** ve **kayan nokta donanımı** vardı; bu durum derlenmiş programlama dilleri fikrine yol açtı, çünkü yorumlama maliyetini saklayacak yer yoktu [PD2, s. 15].

TASARIM ORTAMI

Fortran'ın tasarımı, içinde doğduğu platformun izlerini taşır [PD2, s. 15–16]:

— Bilgisayarlar küçük ve güvenilmezdi.

— Uygulamalar ağırlıklı olarak **bilimseldi**.
— Programlama metodolojileri ve araçları yoktu.
— **Makine verimliliği** en önemli sorundu.
— Dinamik belleğe ihtiyaç yoktu; iyi **dizi işleme** ve **sayma döngüleri** gerekiyordu.
— Ticari iş yazılımı için string işleme, ondalık aritmetik veya güçlü girdi/çıktı yoktu.

Bu liste, Fortran'ın neden bilimsel hesaplamaya özgü doğduğunu açıklar: dil, genel amaçlı bir tasarım değil, **sayısal yöntemlerin ihtiyaçlarına göre** biçimlenmiştir.

FORTRAN I'İN ÖZELLİKLERİ

Fortran I'in gerçekleştirilen ilk sürümü şu özellikleri taşıyordu [PD2, s. 17–18]:

— İsimler **altı karaktere** kadar olabiliyordu.
— **Test-sonrası sayma döngüsü** (`DO`) vardı.
— **Biçimlendirilmiş girdi/çıktı** (formatted I/O) sunuyordu.
— **Kullanıcı tanımlı altprogramlar** vardı.
— **Üçlü seçim ifadesi** (aritmetik `IF`) bulunuyordu.
— Veri tipi bildirimleri yoktu.
— **Ayrı derleme yoktu.**
— Derleyici, **18 iş-yılı** çabadan sonra Nisan 1957'de çıktı.
— 400 satırdan fazla programlar, 704'ün az güvenilirliği yüzünden nadiren doğru derleniyordu.
— Üretilen kod çok hızlıydı ve dil kısa sürede yaygınlaştı.

SÜRÜMLER

SÜRÜM	YIL	GETİRDİĞİ BAŞLICA YENİLİK
Fortran II	1958	Bağımsız derleme; hata düzeltmeleri
Fortran IV	1960–62	Açık tip bildirimleri, mantıksal seçim ifadesi, altprogram adlarının parametre olabilmesi; 1966'da ANSI standardı
Fortran 77	1978	Karakter dizisi işleme, mantıksal döngü kontrolü, `IF-THEN-ELSE`
Fortran 90	—	Modüller, dinamik diziler, işaretçiler, özyineleme, `CASE` , parametre tipi denetimi, bit işlemleri, daha uzun isimler
Fortran 95	—	İşaretçi ve yapılara varsayılan ilk değer, taşınabilirlik; OOP özellikleri eklenmeye çalışıldı
Fortran 2003	—	Fortran 95 çizgisinin sürdürülmesi

Fortran 90'ın öne çıkan farkı, dili yalnızca sayısal hesaplama için değil, **bakımı yapılabilir büyük programlar** için de kullanılabilir kılan yapısal eklemelerdir: modüller, dinamik diziler, işaretçiler, özyineleme ve daha esnek altprogramlar [PD2, s. 22–23].

DEĞERLENDİRME

Fortran'ın başarısı üç noktada toplanır [PD2, s. 24]:

1. **Çok iyi eniyileme yapan derleyiciler** (özellikle 90 öncesi sürümler): bütün değişkenlerin tipleri ve bellekleri çalışma zamanından önce sabitlenir.
2. **Bilgisayarların kullanılma biçimini değiştirmesi**: bilimsel hesaplamanın varsayılan dili olmuştur.
3. **Kullanım kolaylığı**: 1950'lerde makine dilinde yazılan kod daha hızlı çalışsa da Fortran'ın sağladığı üretkenlik kazanmıştır.

Kaynak, Fortran'ı bilgisayar dünyasının **lingua franca**'sı (ortak dili) olarak niteler ve sayısal uygulamalar için uygun bulur [PD2, s. 24]. Örnek program (s. 25), bir listenin ortalamasını hesaplayıp ortalamadan büyük değerleri sayar; Fortran 95'in `Implicit none` , `Integer ::` , `Do ... End Do` , `If ... Then ... End If` yapılarını gösterir.

FORTRAN ÖRNEĞİNİN OKUNMASI

Örnekte `Int_List(99)` bir dizi, `List_Len` okunan eleman sayısıdır. İlk `Do` döngüsü toplamı, ikinci `Do` döngüsü ortalamadan büyük eleman sayısını hesaplar. `Implicit none` bildirimi, değişkenlerin örtük türetilmesini kapatır; bu, Fortran 90 sonrası **güvenilirlik** tercihinin göstergesidir [PD2, s. 25].

2.4 Fonksiyonel programlama: LISP

LISP (LISt Processing — liste işleme), McCarthy tarafından MIT'te IBM 704 için tasarlandı. Yapay zekâ araştırmasının ihtiyaç duyduğu dil iki temel özellik istiyordu: **veriyi dizi yerine liste hâlinde işlemek** ve **sayısal yerine sembolik hesaplama** yapmak. Bu nedenle LISP'in yalnızca iki veri tipi vardır: **atomlar** ve **listeler**. Sözdizimi **lambda kalkülüsüne** dayanır [PD2, s. 26].

KOD VE VERİNİN AYNILIĞI

LISP'in en ayırt edici özelliği, **program kodu ile verinin tam olarak aynı biçimde** olmasıdır. Örneğin `(A B C D)` dizisi [PD2, s. 27]:

— **Veri** olarak okunursa A, B, C ve D verilerini içeren bir listedir.

— **Kod** olarak okunursa A fonksiyonunun B, C ve D parametrelerine uygulanmasıdır.

Bu çift yorum, LISP'e olağanüstü esneklik ve ifade gücü kazandırır; kodun aynı zamanda veri olarak kullanılabilmesi, LISP'i yapay zekâ uygulamalarında rakipsiz kılmıştır [PD2, s. 27]. Kaynak, iki listenin — `(A B C D)` ve `(A (B C) D (E (F G)))` — gösterimini ayrı bir slaytta verir [PD2, s. 29].

LISP'İN ÖZELLİKLERİ VE DEĞERLENDİRMESİ

LISP esas olarak **yorumlayıcı** kullanan bir dildir; derleyici kullanan sürümleri de vardır. Veri tiplemesi bakımından esnektir, nesneye yönelik programlamayı destekler, veritabanı erişimi ve GUI olanağı sunar. İyi belgelenmiştir; **COMMON LISP**, ANSI tarafından standartlaştırılmıştır [PD2, s. 28, 33].

Fonksiyonel programlamanın öncüsü olan LISP'te değişkenlere veya atamaya ihtiyaç yoktur; kontrol, **özyineleme (recursion)** ve **koşullu ifadeler** ile sağlanır. Halen yapay zekânın baskın dillerinden biridir; diferansiyel ve integral hesabında, elektrik devre kuramında, sayısal mantıkta ve yapay zekânın birçok alanında sembolik hesaplamalar için kullanılmıştır. **COMMON LISP** ve **Scheme** güncel lehçelerdir; **ML, Miranda ve Haskell** ilgili dillerdir [PD2, s. 30].

SCHEME VE COMMON LİSP

Scheme, MIT'te 1970'lerin ortalarında geliştirildi. Küçüktür, **statik kapsamı** (static scoping) geniş kullanır ve **fonksiyonları birinci sınıf varlıklar** (first-class entities) olarak ele alır. Basit sözdizimi ve küçük boyutu, onu eğitim amaçlı uygulamalar için ideal kılar [PD2, s. 32]. **Common Lisp** ise LISP'in birkaç lehçesinin özelliklerini tek dilde toplama çabasıdır; büyük ve karmaşıktır [PD2, s. 33].

> KRITIK AYRIM
>
> **Fonksiyonel dil ≠ fonksiyon içeren dil.** LISP'te fonksiyonlar yalnızca çağrılan altprogramlar değil, değer olarak döndürülebilen, başka fonksiyonlara parametre verilebilen varlıklardır. Bu yüzden LISP'te "değişken" ve "atama" merkezî değildir; hesaplama, ifadelerin fonksiyon uygulamasıyla indirgenmesidir.

2.5 Sofistikeliğe doğru ilk adım: ALGOL 60

ALGOL (ALGOrithmic Language), Fortran I'den esinlenmiş olsa da ondan farklı bir soruya yanıt verir: **taşınabilir, evrensel bir algoritma dili** kurmak. Fortran yalnızca IBM 70x için çalışıyordu; geliştirilmekte olan bütün diller belirli makinelere bağlıydı; taşınabilir dil yoktu ve algoritmaları haberleşmek için evrensel bir dil bulunmuyordu [PD2, s. 34].

TASARIM SÜRECİ

ACM ve GAMM, Zürich'te 27 Mayıs–1 Haziran 1958 arasında dört gün görüştü. GAMM (German Society of Applied Mathematics) ve ACM (Association for Computing Machinery), Peter Naur başkanlığında ortak bir komiteyle **International Algorithmic Language (IAL)** adlı dili geliştirdi. Dilin amaçları matematiksel gösterime yakın olmak, algoritma tanımlamak için iyi olmak ve makine koduna çevrilebilmekti [PD2, s. 35].

ALGOL 58

ALGOL 58'in getirdikleri [PD2, s. 36]:

— **Tip kavramı** resmileştirildi.

— İsimler herhangi bir uzunlukta olabiliyordu.

— Diziler herhangi bir sayıda altsimgeye sahip olabiliyordu.

— Parametreler **kip (mode)** ile ayrıldı (`in` ve `out`).

— Altsimgeler köşeli parantez içine yerleştirildi.
— Bileşik ifadeler `begin ... end` ile yazıldı.
— Noktalı virgül ayırıcı oldu.
— Atama için `:=` kullanıldı (`variable := expression`).
— `if` ve `else-if` deyimi vardı.
— **Girdi/çıktı yoktu**; çünkü I/O dili makine bağımlı hâle getirirdi.

ALGOL 58'in uygulanması planlanmadı; yalnızca çeşitleri (MAD, JOVIAL) ortaya çıktı. IBM başlangıçta istekli görünse de 1959 ortasında desteğini çekti [PD2, s. 37].

ALGOL 60'IN YENİLİKLERİ

ALGOL 60, Paris'teki altı günlük toplantı sonucunda ALGOL 58'in değiştirilmesiyle doğdu. Yeni özellikler şunlardı [PD2, s. 38]:

— **Blok yapısı** ve yerel kapsam (local scope).
— İki parametre geçiş yöntemi: **değerle (pass by value)** ve **adla (pass by name)**.
— **Altprogram özyinelemesi (recursion)**.
— **Yığın-dinamik diziler** (stack-dynamic arrays).
— Hâlâ **girdi/çıktı ve karakter dizisi işleme yoktu**.

DEĞERLENDİRME

Başarılar [PD2, s. 39]: yirmi yılı aşkın süre algoritma yayımlamanın standart yolu oldu; sonra gelen bütün zorunlu (imperative) diller ona dayandı; blok kavramı, altprogramlar, özyineleme, yığın ve diziler gibi birçok yeniliğe öncülük etti; **ilk makine-bağımsız dil** oldu; sözdizimi resmî olarak tanımlanan ilk dil oldu (**BNF**); özyineleme ve blok yapısını desteklemek için donanımda yığına izin verilmesini sağlayarak bilgisayar mimarisini etkiledi.
Başarısızlıklar [PD2, s. 40]: özellikle ABD'de yaygınlaşmadı; 1960–70 arasında akademik olarak çok kullanılsa da ticari alanda başarı elde edemedi. Nedenleri: I/O ve karakter kümesi eksikliği programları taşınamaz kıldı; aşırı esneklik anlaşılmayı ve gerçekleştirimi zorlaştırdı; Fortran kökleşmişti ve güçlü kütüphanelere sahipti; resmî sözdizimi tanımı vardı; bilişim sektörünün %80'ine sahip IBM'in desteği yoktu.

> AÇIKLAYICI NOT
> ALGOL 60'ın "başarısızlığı" ticari yaygınlıkla ilgilidir; **tasarım etkisi** açısından tarihin en başarılı dillerinden biridir. Bugün C, Pascal, Ada ve Java'nın blok yapısı, özyineleme ve sözdizimi anlayışı büyük ölçüde ALGOL 60'a dayanır.

2.6 Ticari kayıtları bilgisayara uyarlamak: COBOL

COBOL (COmmon Business Oriented Language), iş dünyasının veri işleme gereksinimleri için doğdu. Geliştirme ortamında üç ayrı gelenek vardı: UNIVAC **FLOW-MATIC**'i, USAF **AIMACO**'yu, IBM ise **COMTRAN**'ı geliştiriyordu; Grace Hopper bu çizginin önemli adlarından biridir [PD2, s. 42].

FLOW-MATIC MİRASI

COBOL, İngilizce sözcük veya cümle yapılarını kullanan, işletmelere yönelik ortak bir dildir; özellikle büyük miktarda bilgi giriş-çıkışı yapılan uygulamalar (stok kontrolü, bordro) için tasarlandı. FLOW-MATIC'ten devraldığı özellikler [PD2, s. 43]:

— İsimler gömülü tirelerle (hyphen) **12 karaktere** kadar çıkabiliyordu.
— Aritmetik operatörler için **İngilizce isimler** kullanılıyordu; aritmetik deyim yoktu.
— **Veri ve kod tamamen ayrıydı.**
— Her ifadede **fiil (verb)** ilk kelimeydi.

TASARIM SÜRECİ

İlk tasarım toplantısı Mayıs 1959'da Pentagon'da yapıldı. Tasarım amaçları [PD2, s. 44]:

— Basit İngilizce gibi görünmeli.
— Daha az güçlü olacağı anlamına gelse bile **kullanımı kolay** olmalı.
— Bilgisayar kullanıcı tabanını genişletmeli.
— Mevcut derleyici problemleriyle kısıtlanmamalı.

Komite üyelerinin tamamı bilgisayar üreticilerinden ve DoD birimlerinden geliyordu. Tasarım problemleri arasında aritmetik ifadelerin olup olmayacağı, altsimgeler ve üreticiler arasındaki anlaşmazlıklar vardı [PD2, s. 44].

KATKILAR VE DEĞERLENDİRME

COBOL'un katkıları [PD2, s. 46–47]: yüksek düzeyli bir dilde **ilk makro** olanağı; **hiyerarşik veri yapıları (records)** ve yan anlamlı isimler (connotative names); iç içe seçim ifadeleri; 30 karaktere kadar uzun isimler ve tireler; **ayrı veri bölümü (data division)**. Program içindeki dil ifadeleri ile fiziksel adresler, verilerin tanımlandığı ve çalıştırılabilir işlemlerin bulunduğu bölümler olarak ikiye ayrılır. Rapor yazdırma, tablo içinde arama ve kolay dosya sıralama rutinleri sayesinde ticari uygulamalarda çok popüler oldu; yapısal özellikleri veritabanı yönetim sistemleriyle birlikte kullanımı kolaylaştırdı. Buna karşın **fonksiyonları desteklemez**.
DoD etkisi ayrı bir başlıktır: COBOL, DoD'nin ihtiyaç duyduğu ilk dildir ve DoD olmasaydı başarısız olacaktı. Halen yaygın kullanılan ticari uygulama dilidir; 1990'larda OOP sürümü üretilmiş ve Amerika büyük veri işlemede hâlâ COBOL kullanmaktadır [PD2, s. 48].

> KRITIK AYRIM
> **COBOL'un okunabilirliği, gücünden ödün vererek elde edilmiştir.** Tasarım amacı "daha az güçlü olsa bile kolay" olduğu için dil, bilimsel hesaplamanın yoğun ifade gücünü değil, iş kayıtlarının **açık ve denetlenebilir** tanımını öne çıkarır. Bu yüzden COBOL örneği, okunabilirlik–ifade gücü ödünleşmesinin ders kitabı örneğidir.

2.7 Zaman paylaşımının başlangıcı: BASIC

BASIC (Beginner's All purpose Symbolic Instruction Code), ABD Dartmouth College'da iki matematikçi **Kemeny ve Kurtz** tarafından, bilimle uğraşmayan kişilerin sayısal hesaplama gereksinimlerini karşılamak için geliştirildi. Kolay öğrenilmesi ve küçük bellekli bilgisayarlarda kullanılabilmesi yaygınlaşmasını sağladı [PD2, s. 50].

TASARIM AMAÇLARI

- Bilim dışı öğrencilerin öğrenmesi ve kullanması kolay bir dil.
- "Güzel ve arkadaşça" olmalı.
- Ödev için **hızlı çalıştırılabilmeli**.
- **Ücretsiz ve kişisel erişim** sağlamalı.
- **Kullanıcının zamanı, bilgisayarın zamanından değerli** olmalı.

BASIC, **zaman paylaşımlı (time-sharing)** ilk yaygın kullanılan dildir; güncel popüler lehçesi **Visual BASIC**'tir [PD2, s. 50].

DİLİN YAPISI

BASIC yalnızca **14 komuta** sahipti (`LET`, `PRINT`, `GOTO` gibi) ve tek veri tipi vardı: sayı (kayan noktalı ve tamsayı). FORTRAN ve ALGOL'den bileşenler aldı: FORTRAN'dan `DO` döngüsünü, ALGOL'den "until" yerine "to" sözcüğünü [PD2, s. 51]. Küçük programları hızlı çalıştırıyor, açık hata mesajları veriyor, kullanıcıyla etkileşimli çalışıyor ve kullanıcıyı işletim sistemi ayrıntılarından koruyordu. Derleyici kullanıyor, programın tamamı makine diline çevrildikten sonra icra ediliyordu [PD2, s. 52].
BASIC'in birçok sürümü oldu; 1989'da nesne yönelimli uyarlama **Visual BASIC**, 1998'de **VB6.0** sunuldu [PD2, s. 52]. Örnek program (s. 54), Fortran örneğinin BASIC'e çevrilmiş hâlidir: `DIM`, `INPUT`, `FOR ... NEXT`, `IF ... THEN ... ELSE`, `PRINT` yapılarıyla aynı ortalama/ sayma algoritmasını kurar.

> AÇIKLAYICI NOT
> BASIC'in "14 komut" sınırı, dilin küçük bellekte ve terminal başında öğretilebilir olmasını sağladı. Ancak aynı sınır, büyük programlarda yapısal soyutlamayı zorlaştırdı; Visual BASIC'in evrimi, bu sınırı OOP ve görsel araçlarla aşma çabasıdır.

2.8 Herkes için her şey: PL/I

PL/I (Programming Language 1), IBM ve SHARE tarafından tasarlandı; IBM'in yeni çıkardığı bilgisayarlarla birlikte gelen bir dildi. COBOL, Fortran IV ve ALGOL 60'tan sonra hem bilimsel hem işletme problemlerine çözüm sağlamak, **kayan nokta ve ondalık veri tiplerini** birlikte desteklemek üzere geliştirildi [PD2, s. 55].

ORTAM VE GEREKÇE

1964'te IBM'in bakış açısından iki kullanıcı grubu vardı [PD2, s. 56]:

KULLANIM	BİLGİSAYAR	DİL	KULLANICI GRUBU
Bilimsel	IBM 1620 ve 7090	FORTRAN	SHARE
Ticari	IBM 1401 ve 7080	COBOL	GUIDE

1963'ten itibaren bilimsel kullanıcılar COBOL'deki gibi ayrıntılı I/O; ticari kullanıcılar ise kayan nokta ve dizi istemeye başladı. Birçok işletme iki tür bilgisayara, iki dile ve iki destek ekibine ihtiyaç duyuyordu; bu pahalıydı. Görünen çözüm, iki uygulama türünü de yapabilen **tek bir bilgisayar ve tek bir dil** tasarlamaktı [PD2, s. 57].

TASARIM VE DEĞERLENDİRME

PL/I, **3 × 3 komitesi** (IBM'den üç, SHARE'den üç üye) tarafından beş ayda tasarlandı. İlk kavram Fortran IV'ün bir uzantısıydı; başlangıçta **NPL (New Programming Language)** adını taşıdı, 1965'te PL/I oldu. En önemli özelliği çeşitli uygulama alanlarında kullanılabilmesiydi; 1970'lerde sayısal ve iş uygulamalarında yaygınlaştı [PD2, s. 58].
İlk PL'in özellikleri [PD2, s. 59]: eşzamanlı çalışabilen altprogramlar; çalışma zamanı hatalarından başka **23 farklı hata tipi**; özyinelemeli prosedürler; işaretçi kullanımı; bir matrisin bir satırına vektör olarak işaret edebilme; bellek gereksinimi yüzünden karmaşıklık. Kaynak, tasarım yönünden dili iyi bulmaz.
Katkılar [PD2, s. 60]: **ilk birim-düzeyli eşzamanlılık**, **ilk istisna işleme**, anahtar-seçmeli özyineleme, **ilk işaretçi veri tipi**, ilk çapraz dizi bölümleri. Zayıflıklar: birçok yeni özellik zayıf tasarlanmıştı; dil aşırı geniş ve karmaşıktı. PL/I, "her şeyi ekleyerek" dil tasarlamanın sınırını gösteren kanonik örnektir.

2.9 İlk iki dinamik dil: APL ve SNOBOL

APL ve SNOBOL, **dinamik tip belirleme (dynamic typing)** ve **dinamik bellek ayrımı (dynamic storage allocation)** ile karakterize edilir. Değişkenlerin tipi yoktur; bir değişkene değer atandığında tip edinir ve bellekte yer ayrılır [PD2, s. 61]. Bu, statik tipli Fortran/ALGOL çizgisinden kesin bir kopuştur.

APL

APL (A Programming Language), 1960'larda IBM'de **Ken Iverson** tarafından bir donanım tanımlama dili olarak tasarlandı. Çok anlamlıdır: hem skaler hem çok boyutlu diziler için birçok operatör sunar. Programların okunması çok zordur; buna karşılık sayısal işlemler ve özellikle matris işlemleri için güçlüdür; tek bir operatörle matris transpozesi alınabilir [PD2, s. 62].
APL'in dezavantajı, ifadeleri oluşturan çok sayıda operatörün karmaşık sözdizimi ve dinamik kaynak gereksinimi nedeniyle bilgisayar kaynaklarının etkin kullanılamamasıdır. Halen kullanımdadır; çok küçük değişiklikler geçirmiştir [PD2, s. 62].

SNOBOL

SNOBOL, Bell Laboratuvarlarında Farber, Griswold ve Polensky tarafından **string ve metin işleme dili** olarak tasarlandı. String desen-eşleştirme (pattern matching) için güçlü operatörleri vardır; alternatif dillere göre yavaştır ve bu yüzden artık yazım editörlerinde kullanılmamaktadır. Yorumlayıcı kullanan bir dildir; veri ve değişkenler üzerinde tip bakımından sınırlama yoktur. Devamı niteliğindeki diller **awk, icon ve perl**'dir; halen bazı metin işleme işlerinde kullanılır [PD2, s. 63].

> KRITIK AYRIM
> **Dinamik tip ≠ zayıf tip.** APL ve SNOBOL'de değişkenlerin bildirilmiş tipi yoktur; tip, atanan değerle belirlenir. Bu, tip denetiminin olmadığı anlamına gelmez; denetim **çalışma zamanına** taşınır. Fortran'ın statik tipleri ise derleme zamanında sabitlenir.

2.10 Veri soyutlamanın başlangıçları: SIMULA 67

SIMULA 67, Norveç'te **Nygaard ve Dahl** tarafından asıl olarak **sistem benzetimi (simulation)** için tasarlandı. ALGOL 60'a (blok yapısı ve kontrol ifadeleri) ve SIMULA I'e dayanır [PD2, s. 64].
Birincil katkıları [PD2, s. 64]:

- **Eşyordam (co-routine):** Daha önce durdurulduğu yerden yeniden çalışmaya başlayan altprogram türü.
- **Sınıf (class):** Hem yerel veri hem de fonksiyonellik içeren yapı.

— **Nesneler ve miras (inheritance).**

— Sınıflar, **veri soyutlamanın (data abstraction)** temelleridir.

SIMULA 67'nin önemi, sınıf kavramını ortaya atarak nesneye yönelik programlamanın kapısını açmasıdır. Örnek (s. 65), `class Lab` içinde `ReadLabel` ve `WriteLabel` prosedürlerini tanımlar; sınıfın hem veriyi (`Nam`, `Street`, `Town`, `County`, `Code`) hem de bu veri üzerindeki işlemleri bir arada tuttuğunu gösterir.

> **AÇIKLAYICI NOT**
> SIMULA'nın "sınıf"ı bugünkü Java/C# sınıfıyla birebir aynı değildir; ancak **veri + davranış birleşimi**, **nesne örnekleri** ve **miras** fikri buradan gelir. Smalltalk ve C++'ın sınıf anlayışı bu hattın devamıdır.

2.11 Ortogonal tasarım: ALGOL 68

ALGOL 68, ALGOL 60'ın gelişmesinden doğdu; ancak onun **üstkümesi değildir**. Dil hiçbir zaman yaygın kullanıma ulaşamasa da birçok yeni fikrin kaynağı oldu [PD2, s. 66].

Tasarımı **ortogonallik (orthogonality)** kavramına dayanır: **az sayıda ilke kavram, az sayıda birleştirici mekanizma**. Amaç, her yapının öngörülebilir biçimde birleşmesidir [PD2, s. 66].

Katkıları [PD2, s. 67]:

— **Kullanıcı tanımlı veri yapılarını** destekleyen ilk dil.

— **Referans tipleri.**

— **Dinamik dizilere** (flex/esnek arrays) izin veren ilk dil.

Olumsuz değerlendirme: ALGOL 60'tan daha az kullanıldı; grameri ve dil yapısı öğrenilmesi oldukça zordu; yalnızca bilimsel uygulamalar hedeflenmişti. Buna karşın sonraki dilleri, özellikle **Pascal, C ve Ada**'yı güçlü biçimde etkiledi [PD2, s. 67].

2.12 ALGOL'lerin ilk torunları

ALGOL dilleri bütün zorunlu (imperative) dilleri etkiledi: **Pascal, C, Modula/Modula 2, Ada, Oberon, C++/Java** ve bir yere kadar **Perl** [PD2, s. 68]. Bu bölüm, tek tek dillerin doğuşunu ve tasarım tercihlerini karşılaştırır.

PASCAL (1971)

Pascal, ALGOL 68 tasarımında çalışan **Niklaus Wirth** tarafından **yapısal programlamayı öğretmek** için tasarlandı. Küçük, basit ve yenilik getirmeyen bir dildi; önceki dillerin özelliklerini bir araya getirdi: COBOL ve PL/I'dan **kayıt** yapısını, ALGOL'den **kullanıcı tanımlı tip** ve **case** deyimini aldı. 1970'lerin ortasından 1990'ların sonuna kadar programlama öğretiminde en yaygın dildi. Basit ve anlaşılır olmasına rağmen bazı özelliklerinin eksikliği ticari kullanımını sınırladı [PD2, s. 69].

C (1972)

C, Bell Laboratuvarlarında **Dennis Ritchie** (ve Ken Thompson) tarafından **sistem programlama** için tasarlandı. Temel olarak **BCPL** ve **B**'den, ayrıca ALGOL 68'den geliştirildi. Esnek yapı ve güçlü operatörler sunar; ancak **tip denetimi zayıftır**. Başlangıçta UNIX üzerinden yayıldı ve birçok uygulama alanı buldu. Kaynak, Pascal ve C'nin programlama literatürüne katkısının az olduğunu; daha çok var olan yapıların bir araya getirildiğini belirtir [PD2, s. 70].

MODULA VE MODULA 2

Modula, yazılım mühendisliği deneyimlerinden doğdu; güçlü **tip ayrımı ve tip denetimi**, dinamik dizi kullanımı (dinamik bellek yönetimi) ve eşzamanlılık özellikleri vardır. **Lilith** adlı bir bilgisayar sistemi için tasarlandı fakat kullanılmadı. **Modula 2**, Pascal'a göre daha başarılı ve esnektir; sözdizimi daha esnektir; gerçek uygulama yazılım sistemleri için geliştirilmiştir. **Soyut veri tipini** destekler, fakat **miras almayı** desteklemez; nesneye yönelim yoktur [PD2, s. 71].

OBERON

Oberon, Modula-2'ye dayanır ama ondan daha basittir. Tipleme, OOP'yi desteklemek için genişletilmiştir; tip ayrımı güçlüdür ve OOP'yi tam olarak destekler. Geleneksel veri türlerini ve kontrol yapılarının çoğunu içerir; dinamik bellek yönetimi vardır. Endüstriyel gücü yüksek bir dildir; veritabanı, ağ haberleşmesi, 3B grafikler ve nesne yönetimi için zengin bir kütüphanesi vardır; soyut veri türlerini destekler [PD2, s. 72].

KRITIK AYRIM

Modula 2 / Oberon / Pascal: Üçü de Wirth çizgisindedir, ancak sınırları farklıdır. Pascal **öğretim** için küçük; Modula 2 **yazılım mühendisliği** ve eşzamanlılık için daha güçlü ama mirassız; Oberon ise **OOP'yi tam destekleyen**, endüstriyel kütüphaneye sahip sade bir dildir.

2.13 Mantık temelli programlama: Prolog

Prolog, Colmerauer ve Roussel (Aix-Marseille Üniversitesi) ile Kowalski'nin (Edinburgh Üniversitesi) yardımıyla geliştirildi. **Biçimsel mantığa (formal logic)** dayanır; mantık yürütme ve ispatlama tekniklerini kullanır; **prosedürel değildir** [PD2, s. 73].
Prolog programı, bir **nesneler kümesi** ile bu nesnelerle ilişkili **hedeflere** nasıl erişilebileceğini tanımlayan **kurallar kümesinden** oluşur. Verilen sorguların doğruluğunu anlamak için **sonuç çıkarma (inference)** kullanan akıllı bir veritabanı sistemi olarak özetlenebilir. 1972'de tanıtılan dilin temel uygulama alanı **doğal dil işlemedir**; veritabanlarından uzman sistemlere kadar çeşitli alanlarda kullanılmıştır. Kaynak, dili çok verimsiz ve uygulama alanını dar bulur [PD2, s. 73].
Örnek (s. 74), aile ilişkileri üzerinden mantık programlamayı gösterir:

```
grandfather(X, Z) :- (mother(X, Y); father(X,Y)), father(Y,Z).
mother(maria, ana).
father(juan, ana).
father(tom, juan).
parent(X, Y) :- mother(X, Y).
parent(X, Y) :- father(X, Y).
grandfather(X, Z) :- parent(X, Y), father(Y,Z).
```

`grandfather(tom, ana)?` sorgusu, kurallar ve gerçekler üzerinden türetilerek yanıtlanır.

AÇIKLAYICI NOT

Prolog'da **kural sırası** ve **arama stratejisi** sonucu etkileyebilir; bu yüzden "bildirimsel" (ne olduğunu söyle) görünümüne karşın uygulama ayrıntıları performansı belirler. Kaynak bu noktayı "çok verimsiz" değerlendirmesiyle birlikte anar [PD2, s. 73].

2.14 Tarihin en büyük tasarım çabası: Ada

Ada, ABD Savunma Bakanlığı (DoD) tarafından **gömülü sistemler** için 1974–1979 arasında geliştirildi. Yüzlerce insan, büyük para ve yaklaşık **sekiz yıl** süren tasarım çabasıydı. Gereksinim belgeleri sırasıyla **Strawman** (Nisan 1975), **Woodman** (Ağustos 1975), **Tinman** (1976), **Ironman** (1977) ve **Steelman** (1978) adlarını taşır. Dil, ilk programcı olarak bilinen **Augusta Ada Byron**'dan (1815–1851) sonra adlandırıldı [PD2, s. 75].

KATKILAR VE ELEŞTİRİLER

Katkılar [PD2, s. 76–77]:

- **Paketler** ve veri soyutlama desteği.
- Ayrıntılı **istisna işleme (exception handling)**.
- **Generic (şablon/soysal) program birimleri** ile yazılımın yeniden kullanımı.
- **Eşzamanlılık**: görevleme (tasking) modeli ve **rendezvous** mekanizması.
- Blok yapısı ve büyük ölçekli yazılımlara uygunluk.

Eleştiriler: rekabetçi tasarım süreci; yazılım mühendisliği ve dil tasarımı hakkında sonradan bilinen her şeyi içermesi; ilk derleyicilerin çok zor olması. İlk gerçekten kullanılabilir derleyici, dil tasarımının tamamlanmasından yaklaşık **beş yıl** sonra geldi. Dil çok geniş ve karmaşıktı (özellikle yazım kuralları); erken derleyiciler verimsiz kod üretiyordu. En çok **gömülü sistemlerde** başarılı oldu; veri tipleri bakımından zengin ve çok-işlem özelliklidir [PD2, s. 76–77].

ADA 95

Ada 95 (çalışmalar 1988'de başladı) şunları getirdi [PD2, s. 78]: tip türetme üzerinden OOP desteği; paylaşılan veri için daha iyi denetim; yeni eşzamanlılık özellikleri; daha esnek kütüphaneler; altprogramların dinamik kapsam bağlama kurallarına göre çağrılması. Popülerliği azaldı; çünkü DoD artık zorunlu kullanım talep etmiyordu ve C++ yükseliyordu.

> KRITIK AYRIM
> **Ada'nın "başarısızlığı" teknik değil, benimsenme sorunudur.** Dil, güvenlik ve eşzamanlılık açısından güçlüydü; ancak devasa tasarım, pahalı derleyiciler ve zorunlu kullanımın kalkması yaygınlaşmayı sınırladı. Gömülü sistemlerdeki etkisi bugün de sürer.

2.15 Nesneye dayalı programlama: Smalltalk

Smalltalk, Xerox PARC'ta önce **Alan Kay**, sonra **Adele Goldberg** tarafından geliştirildi. Bir nesneye dayalı dilin **ilk tamamen gerçekleştirilmiş** örneğidir: veri soyutlama, miras (inheritance) ve dinamik tip bağlama (binding) bir arada. Grafiksel kullanıcı arayüzü tasarımına öncülük etti; yalnızca bir dil değil, aynı zamanda bir **yazılım geliştirme aracıdır** ve OOP'yi yükseltmiştir [PD2, s. 80].
İlk olarak **Smalltalk-80** sürümü Xerox dışında ticari yazılımlarda kullanıldı. Smalltalk, tamamen nesne yönelimli ve ticari ilk programlama dilidir. Geliştirme ortamı (tarayıcı, editör, hata ayıklayıcı, açık kaynak kod) nesne yönelimli altyapısı sayesinde çok gelişmiştir; açık kaynak kod onu iyi bir eğitim ortamı yapar. Sözdizimi diğer dillere göre çok basittir; Java gibi geniş ve sürekli genişleyen bir kütüphaneye sahiptir ve platform bağımsızdır [PD2, s. 81].
Smalltalk'ta OOP'nin üç temel karakteristiği eksiksiz bulunur: **veri soyutlama**, **kalıtım (inheritance)** ve **dinamik bağlama (dynamic binding)** [PD2, s. 81].

> AÇIKLAYICI NOT
> Smalltalk'un "her şey nesnedir" ilkesi, C++ ve Java'daki "ilkel tipler" istisnasını ortadan kaldırır. Bu yüzden Smalltalk, OOP'nin **saf** biçimi olarak anılır; Java'nın ilkel tipleri ise performans için yapılmış bir ödündür.

2.16 Zorunlu ve nesneye dayalı özellikleri birleştirmek: C++

C++, Bell Labs'da **Stroustrup** tarafından 1980'de geliştirildi. **C** ve **SIMULA 67**'den türedi; nesneye dayalı olanaklarının bir kısmı SIMULA 67'den alındı. **İstisna yakalama (exception handling)** sağlar; hem prosedürel hem nesneye yönelik programlamayı desteklediği için geniş ve karmaşık bir dildir. Popülerliği OOP ile birlikte hızla arttı; ANSI standardı **Kasım 1997**'de onaylandı [PD2, s. 82].
Öne çıkan özellikler [PD2, s. 83]:

- **Çoklu miras alma** desteği.
- Operatör ve metotların **üst üste bindirilmesi (overloading)**.
- Sınıf ve metotların **template** edilebilmesi.
- Güçlü tip ayrımı, dinamik bellek yönetimi, hazır şablonlar ve **çok biçimlilik (polymorphism)**.
- C'nin özelliklerinin çoğunu koruması.
- PL/I gibi geniş ve karmaşık olması; Ada ve Java'dan daha az güvenli sayılması.

Microsoft'un **Managed C++** sürümü 2002'de .NET ile çıktı; delegeler (delegates) ve arayüzler (interfaces) getirdi, çoklu miras yoktu [PD2, s. 83].

EİFFEL VE DELPHİ

Eiffel (Bertrand Meyer, 1992), doğrudan başka bir dilden gelmeyen, zorunlu ve OO özellikleri birleştiren melez bir dildir. C++'tan küçük ve basittir ama hâlâ güçlüdür; C++ kadar popüler değildir çünkü C++ hayranlarının çoğu önceden C programcısıydı. Soyut veri yapılarını, kalıtımı ve dinamik bildirimleri destekler; altprogram ile çağırıcı arasındaki iletişimde **bildirimleri (assertions)** kullanır [PD2, s. 84].
Delphi (Borland), Pascal artı OOP özellikleridir; C++'tan daha zarif, daha az karmaşık ve güvenlidir. Pascal'dan türediği için dizi elemanı denetiminde, işaretçi aritmetiğinde ve tip zorlamalarında C/C++'tan daha güvenlidir. Kullanıcı tanımlı operatörlere, generic altprogramlara ve parametreleştirilmiş sınıflara izin vermez; GUI sağlar [PD2, s. 85].

2.17 Zorunlu tabanlı nesneye dayalı dil: Java

Java, 1990'ların başında Sun'da geliştirildi. C ve C++ gömülü elektronik aygıtlar için yeterince memnun edici değildi. Java, C++ temellidir ama önemli ölçüde basitleştirilmiştir: `struct`, `union`, `enum`, işaretçi aritmetiği ve C++'ın atama zorlamalarının yarısı yoktur; yalnızca OOP'yi destekler; referanslar vardır, işaretçiler yoktur; appletler ve bir eşzamanlılık biçimi için destek içerir [PD2, s. 86].

DEĞERLENDİRME

Java, C++'ın güvensiz özelliklerini elemiş; eşzamanlılık, applet kitaplıkları, GUI ve veritabanı erişimi sunmuştur. **Taşınabilirlik** Java Sanal Makinesi (JVM) ve JIT derleyicilerle sağlanır; WWW sayfalarında yaygın kullanılır ve diğer alanlarda kullanımı hızla artar [PD2, s. 87].
Dilin yapısal özellikleri [PD2, s. 88–90]:

- Daha küçük, daha basit ve daha güvenilir bir dil hedefi.
- Basit, taşınabilir ve nesneye yönelik yapı.
- Miras alma, çok biçimlilik, güçlü tip denetimi, eşzamanlılık denetimi, dinamik yüklenebilen kütüphaneler, diziler, string işlemleri ve standart kütüphane.
- Temel yapısal bileşen **sınıf**; bütün veri ve metotlar bir sınıfla ilişkilidir; global veri veya fonksiyon yoktur.
- C++'taki çoklu miras, operatör aşırı yüklemesi ve makro önişlemcisi yoktur; şablon yapıları da yoktur (ihtiyaç minimuma inmiştir).
- İşaretçi yoktur; bütün nesne sınıfları `Object` kök sınıfından miras alır.
- `record`, `union`, `enum` tipleri yoktur; prosedürel programlamayı desteklemez.
- Yalnızca tekli miras; gelişmiş GUI; iplik (threads) ile eşzamanlılık; **çöp toplama (garbage collection)**; güçlü tip dönüşümü.

ÇALIŞMA BİÇİMİ

Java tipik olarak platformdan bağımsız **bytecode** biçiminde derlenir. Bytecode, **Java Sanal Makinesi** adı verilen yorumlayıcı tarafından kullanılacağı platformun makine diline çevrilir. Derlenmiş Java sınıflarının taşınabilirliğini garantileyen özellik, **.class** biçiminin kesin olarak tanımlanmış olmasıdır. **Applet** adı verilen gömülü programlar web tarayıcılarında çalışabilir; Java internet ve web programcılığında yaygınlaşmıştır [PD2, s. 90].

> **KRITIK AYRIM**
> **Java ≠ JavaScript.** Ad benzerliği dışında ilişki yoktur; JavaScript betik dilidir ve Java'nın veri tipi katılığına karşı dinamik tipleme kullanır (bkz. 2.18). Java'da `Object` kalıtımı ve JVM bytecode modeli belirleyicidir.

2.18 Ağ için betik dilleri

Betik dili (scripting language), çalıştırılacak komutları içeren bir **yazı dosyası (script file)** üzerinden çalışır; `sh`, `awk`, `tcl/tk` diğer örneklerdir [PD2, s. 92]. Bu bölümdeki diller, web ve metin işleme alanında farklı tasarım tercihlerini temsil eder.

PERL

Perl, ALGOL ile yalnızca C üzerinden ilişkilidir; bir betik dilidir. Larry Wall tarafından geliştirilmiştir. Değişkenleri **statik tiplidir** ve örtülü (implicitly) tanımlanır; üç farklı **isim alanı (namespace)** bir değişkenin adının ilk karakteriyle gösterilir. Güçlü fakat tehlikeli sayılır; genel amaçlı dil olarak yaygınlaşmış, web CGI programlamada ve UNIX sistem yönetiminde kullanılmıştır [PD2, s. 92].

JAVASCRİPT

JavaScript, Netscape ve Sun Microsystems ortaklığıyla 1995'te üretildi; orijinal adı **LiveScript**'tir. Web programlamada **istemci tarafında (client side)** dinamik HTML belgeleri oluşturmak için kullanılır; tamamen yorumlayıcıdır. Java ile yalnızca benzer sözdizimi nedeniyle ilgilidir. İçinde web sayfalarının çeşitli kısımlarını, özellikle HTML'i işleyecek ve denetleyecek bileşenler vardır. Java'nın katı derleme zamanı tiplerine karşılık JavaScript **dinamik tipleme** açısından toleranslıdır; Java'nın HTML desteği zayıfken JavaScript'in desteği tamdır. Nesneye yönelik ve blok yapısal bir dildir [PD2, s. 93].

PHP

PHP (Personal Home Page / PHP: Hypertext Preprocessor), Rasmus Lerdorf tarafından 1994'te tasarlandı. **Sunucu tarafı (server side)** web uygulamaları için kullanılır; genellikle form işleme ve veritabanı erişiminde kullanılan, HTML içine gömülü bir betik dilidir; çıktı olarak HTML üretir. HTML belgesi, okuyucuyu hiper bağlantılar boyunca başka belgelere götürecek biçimde programlanabilir. Gömülü PHP kodu, belge tarayıcı tarafından istendiğinde web sunucusunda yorumlanır. PHP'nin dizi yapısı JavaScript ve Perl dizilerinin bir birleşimidir; form erişimi oldukça kolaydır ve dil tamamen yorumlayıcıdır [PD2, s. 94].

PYTHON, LUA VE RUBY

Python, nesne tabanlı yorumlayıcıya sahip bir metin dilidir; tipi denetlenir ama dinamik yazılır; CGI programlama ve form işleme için kullanılır; listeleri, değişken gruplarını ve karmaları destekler [PD2, s. 95].
Lua, benzer biçimde nesne tabanlı yorumlayıcıya sahip bir metin dilidir; tipi denetlenir ama dinamik yazılır; CGI ve form işlemede kullanılır; listeler, değişken grupları ve karmaların hepsini **tek veri yapısı olan tablolar** üzerinden yapar ve kolayca genişletilebilir [PD2, s. 95].
Ruby, Yukihiro Matsumoto ("Matz") tarafından Japonya'da tasarlandı; Perl ve Python için bir yedek dil olarak başladı; **saf nesne yönelimli** bir betik dilidir. Bütün veriler nesnedir; birçok operatör kullanıcı kodu tarafından yeniden tanımlanabilen metotlar olarak uygulanır; saf yorumlayıcıdır [PD2, s. 96].

> AÇIKLAYICI NOT
> Betik dilleri "derlenmez" diye düşünmek yanıltıcıdır: JavaScript ve PHP çoğunlukla yorumlanır, ancak modern uygulamalarda ara biçim/JIT katmanları görülebilir. Kaynak, bu dilleri **yorumlayıcı** ve ** dinamik tipleme** ekseninde sınıflar [PD2, s. 93–96].

2.19 Yeni milenyum için C temelli dil: C#

C#, .NET geliştirme platformunun bir parçasıdır; **C++, Java ve Delphi** temellidir. **Bileşen temelli (component-based)** yazılım geliştirme için dil sağlar. Bütün .NET dilleri (C#, Visual Basic .NET, Managed C++, J#.NET ve JScript.NET) **Ortak Tip Sistemi**'ni (Common Type System, CTS) kullanır; bu ortak bir sınıf kütüphanesi sağlar. C# yaygın kullanıma ulaşmıştır [PD2, s. 97].
C#'ın özellikleri [PD2, s. 98–99]:

- C ve C++ dil ailesinin ilk **bileşen yönelimli** dilidir.
- C ve C++'tan derlenmiş, basit, modern, nesne yönelimli ve **tür güvenli** bir dildir.
- Yüksek başarımlı **Common Language Runtime (CLR)**; yürütme motoru, çöp toplayıcı, anında derleme (JIT), güvenlik sistemi ve zengin sınıf çerçevesi (.NET Framework) içerir.
- CLR, birden çok dil desteği için tasarlanmıştır; **Common Language Specification (CLS)** dil işlevselliğinin ortak düzeyini tanımlar.
- Otomatik bellek yönetimi kullanır; tür sistemi işaretçi türlerine ve nesne adreslerinin doğrudan değiştirilmesine de izin verir.
- Tür sistemi bileşiktir; **her şey bir nesnedir**. **Kutulama ve kutuyu açma (boxing/unboxing)** ile değer türleri ve başvuru türleri arasındaki açık kapatılır.

> KRITIK AYRIM
> **C# / Java:** İkisi de C temelli, yönetilen (managed) ve OO'dir. C#'ta `struct`, `enum`, operatör aşırı yüklemesi ve `unsafe` işaretçiler vardır; Java'da bunlar yoktur, buna karşılık Java'nın JVM taşınabilirliği ve çöp toplama modeli belirleyicidir. C#, .NET'in **bileşen** ve **diller arası birlikte çalışma** hedefiyle ayrılır.

2.20 İşaretleme/programlama hibrit diller

XSLT: XML (eXtensible Markup Language — genişletilebilir işaretleme dili) bir **metamarkup** dilidir. **XSL Transformation** (XSLT), XML belgelerini görüntülenebilmesi için dönüştürür; içinde döngüler gibi **programlama yapıları** bulunur [PD2, s. 100]. Bu yönüyle XSLT, işaretleme ile programlamanın hibrit örneğidir.
JSP: Java Server Pages (Java Sunucu Sayfaları), dinamik web belgelerini destekleyen teknolojiler koleksiyonudur. Bir **servlet**, web sunucusuna ait bir Java programıdır; servletin çıktısı tarayıcıda görüntülenir [PD2, s. 100]. JSP, HTML içine Java kodunun gömülmesiyle aynı hibrit mantığı sürdürür.

> AÇIKLAYICI NOT
> Kaynak, bu iki teknolojiyi "hibrit" sayarken **dilin kendisi** ile **belge dönüştürme/şablon sistemini** birbirinden ayırır. XSLT bir dönüştürme dilidir; JSP ise sunucu tarafında çalışan bir şablon teknolojisidir. Sınavda ikisinin de **işaretleme + programlama** kesişiminde durduğu söylenmelidir.

BÖLÜM 2'NİN SENTEZİ

PD2'nin ana çizgisi, dil tasarımının **tek bir doğruya** değil, **ödünleşmelere** dayandığını gösterir [PD2, s. 102]:

- **Makine verimliliği ↔ insan üretkenliği:** Fortran ve C hızlı kod üretir; BASIC ve COBOL okunabilirliği öne çıkarır.
- **Güç ↔ basitlik:** PL/I ve Ada "her şeyi" ekler, karmaşıklaşır; Pascal ve Oberon sade kalmayı seçer.

— **Taşınabilirlik ↔ donanım erişimi:** ALGOL 60 ve Java taşınabilirliği; C sistem programlamayı öne çıkarır.
— **Statik ↔ dinamik:** Fortran/ALGOL statik tiplidir; APL, SNOBOL ve betik dilleri dinamik tiplemeyi seçer.
— **Prosedürel ↔ bildirimsel:** C ve Pascal prosedürel; LISP fonksiyonel; Prolog mantıksaldır.

C–G · ORTAK BÖLÜMLER
SÖZLÜK · PD2

C

Kavramlar ve tanımlar

PD2'nin sınav açısından kritik terimleri; dillerin tasarım amaçları ve birbirinden ayrıldığı noktalar.

DİL VE TASARIM

Plankalkül: Konrad Zuse'nin ilk yüksek düzeyli dili; kayan nokta, dizi ve kayıtları öngördü; 1972'de yayımlandı, derleyicisi 1998'de çıktı. [PD2, s. 7–10]

Sözdekod (pseudocode): Makine kodunun okunaksızlığını azaltan, insan için kısa komutlarla çalışan ara çeviri katmanı. [PD2, s. 11–13]

Fortran: IBM 704 için Backus'un bilimsel hesaplama dili; derlenen ilk yaygın yüksek düzeyli dil. [PD2, s. 15–24]

LISP: McCarthy'nin liste işleme ve sembolik hesaplama dili; kod ve veri aynı biçimdedir. [PD2, s. 26–30]

ALGOL 60: Taşınabilir algoritma dili; blok yapısı, özyineleme, değer/adla parametre geçişi ve BNF ile tanımlanan ilk sözdizimi. [PD2, s. 34–40]

COBOL: İş kayıtları için İngilizce benzeri, veri ve kodu ayıran dil; DoD desteğiyle yaygınlaştı. [PD2, s. 42–48]

DİLLER VE YENİLİKLER

BASIC: Zaman paylaşımı için küçük, öğretilebilir dil; 14 komut ve tek sayı tipi. [PD2, s. 50–52]

PL/I: Bilimsel ve ticari uygulamaları tek dilde birleştirme girişimi; ilk istisna işleme, işaretçi ve eşzamanlılık. [PD2, s. 55–60]

APL / SNOBOL: Dinamik tipli ilk diller; APL dizi/matris, SNOBOL string desen eşleştirme. [PD2, s. 61–63]

SIMULA 67: Sınıf, nesne, miras ve eşyordam kavramlarının kaynağı; veri soyutlama. [PD2, s. 64–65]

ALGOL 68: Ortogonal tasarım; kullanıcı tanımlı veri yapıları, referans tipleri, dinamik diziler. [PD2, s. 66–67]

DİLLER VE PARADİGMALAR

Prolog: Mantık temelli, kural ve gerçeklerden sonuç çıkaran dil; doğal dil işleme. [PD2, s. 73–74]

Ada: Gömülü sistemler için paket, generic, istisna ve görevleme (tasking) sunan dil. [PD2, s. 75–79]

Smalltalk: İlk tamamen OOP dil; veri soyutlama, miras ve dinamik bağlama. [PD2, s. 80–81]

C++: C + SIMULA sınıfları; çoklu miras, şablon, operatör aşırı yükleme. [PD2, s. 82–85]

Java: C++'ın sadeleştirilmiş, taşınabilir (JVM/bytecode) ve yalnız OOP sürümü. [PD2, s. 86–90]

C#: .NET/CLR üzerinde bileşen yönelimli, tür güvenli dil; CTS ve CLS. [PD2, s. 97–99]

Betik dili: Yorumlanan, genellikle dinamik tipli, metin/web odaklı dil (Perl, JavaScript, PHP, Python, Lua, Ruby). [PD2, s. 92–96]

XSLT / JSP: İşaretleme ile programlamayı birleştiren dönüştürme/şablon teknolojileri. [PD2, s. 100]

SIK KARIŞTIRILAN AYRIMLAR

Fortran / ALGOL 60: Bilimsel hesaplama ve IBM uyumu / taşınabilir algoritma dili ve BNF.

COBOL / PL/I: Ticari kayıt odaklı, veri-kod ayrımı / bilimsel + ticari birleşim, aşırı genişlik.

LISP / Prolog: Fonksiyonel, liste ve özyineleme / mantıksal, kural ve sorgu.

Simula / Smalltalk: Sınıf ve benzetim / saf nesne ve geliştirme ortamı.

C++ / Java: Çoklu miras, işaretçi, şablon / tekli miras, referans, JVM.

C# / Java: CLR/CTS, bileşen, `struct` ve `unsafe` / JVM, yalnız OOP, taşınabilirlik.

D

Yöntemler ve süreçler

Dillerin tarih içindeki gelişimini okumayı kolaylaştıran karşılaştırma eksenleri ve tasarım–değerlendirme akışı.

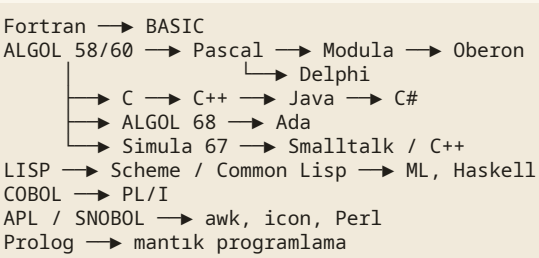
D.1 BİR DİLİ TARİHSEL OLARAK OKUMA YÖNTEMİ

1. **Sorunu belirle:** Dil hangi donanım, uygulama ve kullanıcı grubu için doğdu? [PD2, s. 15, 42, 57]
2. **Tasarım amaçlarını çıkar:** Dil neyi öne çıkarıyor, neyden ödün veriyor? [PD2, s. 35, 44, 50]
3. **Yenilikleri ayır:** İlk kez hangi yapı bu dilde görüldü? [PD2, s. 60, 64, 67]
4. **Değerlendirmeyi oku:** Kaynak dilin başarı ve başarısızlığını hangi gerekçelerle açıklıyor? [PD2, s. 39–40, 76–77]
5. **Soyağacına yerleştir:** Dil hangi dilden türedi, hangi dilleri etkiledi? [PD2, s. 68, 82, 86]

D.2 TASARIM ÖDÜNLEŞMELERİ TABLOSU

EKSEN	BİR UÇ	DİĞER UÇ
Okunabilirlik / güç	COBOL, BASIC	PL/I, C++
Taşınabilirlik / donanım	ALGOL 60, Java	C, assembly
Statik / dinamik	Fortran, Pascal	APL, SNOBOL, betik dilleri
Prosedürel / bildirimsel	C, Pascal, Ada	LISP, Prolog
Basitlik / kapsam	Pascal, Oberon	PL/I, Ada

D.3 DİL AİLELERİ



AÇIKLAYICI NOT

Bu şema kaynak slaytlardaki “ALGOL dilleri bütün zorunlu dilleri etkiledi” ve “C++ C ve SIMULA 67’den geliştirildi” ifadelerine dayanır [PD2, s. 68, 82]. Oklar **tam türetme** değil, **etki/soy** ilişkisini gösterir.

C–G · ORTAK BÖLÜMLER
SINAV BİLGİLERİ

E

Formüller, kurallar ve olgular

PD2'de geçen tarihler, sürümler, sayısal sınırlar ve dil özellikleri. Yalnızca kaynakta verilen bilgiler; ek açıklamalar etiketlidir.

E.1 TARİHLER VE SÜRÜMLER

OLGU	DEĞER / İÇERİK	KAYNAK
ENIAC	1943; ilk programlama dili ENIAC Programlama Dili	PD2, s. 5–6
Plankalkül	1972'de yayım; ilk derleyici 1998; bağımsız uygulama 2000	PD2, s. 10
Short Code	1949, Mauchly, BINAC	PD2, s. 12
Speedcoding	1954, Backus, IBM 701	PD2, s. 13
Fortran 0 / I	1954 tasarım / 1957 ticari sürüm	PD2, s. 15
Fortran II / IV / 77 / 90 / 95 / 2003	1958 / 1960–62 / 1978 / — / — / —	PD2, s. 19–23
ALGOL 58 toplantısı	27 Mayıs–1 Haziran 1958, Zürich	PD2, s. 35
COBOL tasarım toplantısı	Mayıs 1959, Pentagon	PD2, s. 44
BASIC	Dartmouth; Kemeny ve Kurtz	PD2, s. 50
PL/I	3 × 3 komitesi, beş ay; 1965'te PL/I adı	PD2, s. 58
Prolog	1972'de tanıtıldı	PD2, s. 73
Ada	1974–1979; Strawman 1975, Woodman 1975, Tinman 1976, Ironman 1977, Steelman 1978	PD2, s. 75
Ada 95	Çalışmalar 1988'de başladı	PD2, s. 78
C++	1980, Stroustrup; ANSI standardı Kasım 1997	PD2, s. 82
Java	1990'ların başı, Sun	PD2, s. 86
JavaScript	1995; özgün adı LiveScript	PD2, s. 93
PHP	1994, Rasmus Lerdorf	PD2, s. 94
C#	.NET platformu; 2002 Managed C++	PD2, s. 83, 97

E.2 SAYISAL SINIRLAR VE ÖZELLİKLER

OLGU	DEĞER
Fortran I isim uzunluğu	6 karakter [PD2, s. 17]
Fortran I derleyici emeği	18 iş-yılı [PD2, s. 18]
Speedcoding kullanıcı belleği	700 kelime [PD2, s. 13]
COBOL isim uzunluğu	FLOW-MATIC'te 12, COBOL'de 30 karakter [PD2, s. 43, 46]
BASIC komut sayısı	14 [PD2, s. 51]
PL/I hata tipi	Çalışma zamanı dışında 23 tip [PD2, s. 59]
Ada tasarım süresi	Yaklaşık 8 yıl [PD2, s. 75]

E.3 KAYNAK İÇİ TUTARSIZLIKLAR VE YAZIM NOTLARI

— **"Richie" / "Ritchie":** Kaynakta C'nin geliştiricisi "Dennis Richie" olarak yazılmıştır; yaygın yazım **Dennis Ritchie**'dir [PD2, s. 70].

— **"PL/I" adlandırması:** Başlangıçta NPL (New Programming Language) olan ad 1965'te PL/I olmuştur [PD2, s. 58].

— **“Java” örneği:** Kaynak slaytta `g.drawString` satırı bölünmüş görünür; kılavuzda anlamı korunmuş, kod birebir çoğaltılmamıştır [PD2, s. 91].

— **2.9 başlığı:** Kaynakta “İlk İki Dinamik Dil: APL ve SNOBOL” denir; dinamik tip kavramı bu iki dille sınırlı değildir, ancak kılavuz kaynak kapsamını izler [PD2, s. 61].

C–G · ORTAK BÖLÜMLER
BÖLÜMLER ARASI AĞ

F

Bağlantılar ve karşılaştırmalar

PD2'nin PD0–PD1 ve sonraki bölümlerle bağlantısı; dil tarihinin kavram haritasına nasıl oturduğu.

F.1 PD0–PD1 → PD2: KAVRAMDAN TARİHE

PD0–PD1'DEKİ KAVRAM	PD2'DEKİ KARŞILIĞI	İLİŞKİ
Programlama dili tanımı (PD0 0.2)	Plankalkül, Fortran, ALGOL 60	Tanım, tarihsel örneklerle somutlaşır.
Derleme/yorumlama (PD0 0.4)	Fortran derleyicisi; LISP yorumlayıcısı; Java bytecode	Aynı çevrim yöntemleri farklı dillerde görülür.
Okunabilirlik–yazılabilirlik–maliyet (PD1 1.3)	COBOL, BASIC, PL/I, APL	Tarihsel tercihler değerlendirme ölçütleriyle açıklanır.
Paradigmalar (PD1 1.5)	LISP, Prolog, Smalltalk, C++	Paradigma kavramı somut dillere bağlanır.
Von Neumann mimarisi (PD1 1.4)	Fortran, C, imperative diller	Makine modeli, zorunlu dillerin baskınlığını açıklar.

F.2 SONRAKİ BÖLÜMLERLE BAĞLANTI

- **PD3 (sözdizimi ve anlambilim):** ALGOL 60'ın BNF ile tanımlanması, bu bölümün en güçlü köprüsüdür; PD3 gramerleri, türetmeleri ve anlam tanımlarını biçimselleştirir.
- **PD4 (sözcüksel/sözdizimsel analiz):** Fortran ve ALGOL'ün derleyici mirası, ayrıştırma algoritmalarının tarihsel zeminini verir.
- **PD5 (adlar, bağlama, kapsam):** ALGOL 60'ın blok yapısı ve statik kapsamı, isim/görünürlük konularının başlangıcıdır.
- **PD6 (veri tipleri):** Fortran'ın tipleri, APL/SNOBOL'ün dinamik tipleri ve C'nin zayıf tip denetimi doğrudan PD6'ya bağlanır.
- **PD7 (kontrol yapıları):** BASIC'in `FOR/NEXT`, ALGOL'ün blok ve özyinelemesi, C'nin kontrol deyimleri.
- **PD8 (altprogramlar):** Fortran altprogramları, ALGOL'ün değer/adla geçişi, Ada'nın generic birimleri ve Java metotları.

> **BU KILAVUZUN YORUMU**
>
> PD2'nin ortak ekseni **soyutlama düzeyidir**: Plankalkül donanımdan bağımsızlaşmayı, Fortran bilimsel hesaplamayı, COBOL kayıtları, LISP sembolleri, SIMULA sınıfları, Smalltalk nesneleri, Java sanal makineyi soyutlar. Her yeni soyutlama, bir öncekinin sınırını görünür kılar; dil tarihi bu yüzden ilerleme kadar **katmanlaşma** tarihidir.

G

Kapsam denetimi

Bu bölüm, PD2'nin hangi sayfalarının kapsandığını, hangi düşük metinli slaytların görsel olarak incelendiğini ve neyin bilinçli olarak dışarıda bırakıldığını raporlar.

G.1 Kaynak bazında kapsam

KAYNAK	SLAYT	KAPSANAN BÖLÜMLER VE SAYFA ARALIKLARI	DURUM
PD2	103	2.1 (s. 7–10) · 2.2 (s. 11–14) · 2.3 (s. 15–25) · 2.4 (s. 26–33) · 2.5 (s. 34–41) · 2.6 (s. 42–49) · 2.7 (s. 50–54) · 2.8 (s. 55–60) · 2.9 (s. 61–63) · 2.10 (s. 64–65) · 2.11 (s. 66–67) · 2.12 (s. 68–72) · 2.13 (s. 73–74) · 2.14 (s. 75–79) · 2.15 (s. 80–81) · 2.16 (s. 82–85) · 2.17 (s. 86–91) · 2.18 (s. 92–96) · 2.19 (s. 97–99) · 2.20 (s. 100) · özet (s. 102) · kaynaklar (s. 103)	Tam kapsam

G.2 Metin katmanı olmayan / düşük metinli slaytlar

PD2'de **9** düşük metinli slayt vardır: **1, 5, 19, 21, 29, 33, 45, 53, 101**. Bunlar diyagram, kod görseli veya boş slayt türündedir. Aşağıdaki tablo, bu slaytların nasıl ele alındığını özetler.

SAYFA	İÇERİK / GÖRSEL	KULLANIM
1	Bölüm kapağı	İçindekilere ve bölüm açılışına yansıtıldı
5	Yaygın dillerin soyağacı	D.3 dil aileleri şeması ve Bölümün haritası ile temsil edildi
19, 21	Fortran II / Fortran 77	Metin katmanı yetersiz; sürüm tablosunda özetlendi
29, 33, 53	LISP liste gösterimi, COMMON LISP, BASIC şeceresi	Metin olarak açıklandı; D.3 şemasında temsil edildi
45	COBOL görseli	Boş metin; COBOL örneği s. 49'dan özetlendi
101	Boş slayt	Kapsam dışı; metin yok

G.3 Bilinçli dışarıda bırakılanlar ve sonuç

— **Kapak ve konu listeleri** (s. 1–4) içindekilere yansıtıldı; her madde ayrıca açılmadı.

— **COBOL örneği** (s. 49) iki sütunlu, çok uzun bir kod görselidir; kılavuzda dilin yapısı ve `IDENTIFICATION`, `ENVIRONMENT`, `DATA`, `PROCEDURE` bölümleri kavramsal olarak açıklandı, kod birebir çoğaltılmadı.

— **Fortran, BASIC, LISP, Simula, Prolog, Ada ve Java örnekleri** (s. 25, 31, 54, 65, 74, 79, 91) yalnızca ayırt edici yapılarıyla özetlendi; tam kod listeleri sayfa sınırı nedeniyle alınmadı.

— **Dekoratif görseller** ve tekrarlanan kaynak listeleri (s. 103) yeniden üretilmedi; kaynakça listesi birleştirildi.

— **Ödev slaytı yoktur**; PD2'de ödev sayfası bulunmaz.

Sonuç: PD2'nin **20/20 alt bölümü** ve **103/103 slaytı** kapsandı. Düşük metinli 9 slaytın tamamı incelendi; erişilemeyen sayfa tespit edilmedi. Otomatik başlık denetimi `build --check` ile, tablo/şekil/örnek denetimi bu bölümde elle yapılmıştır.